

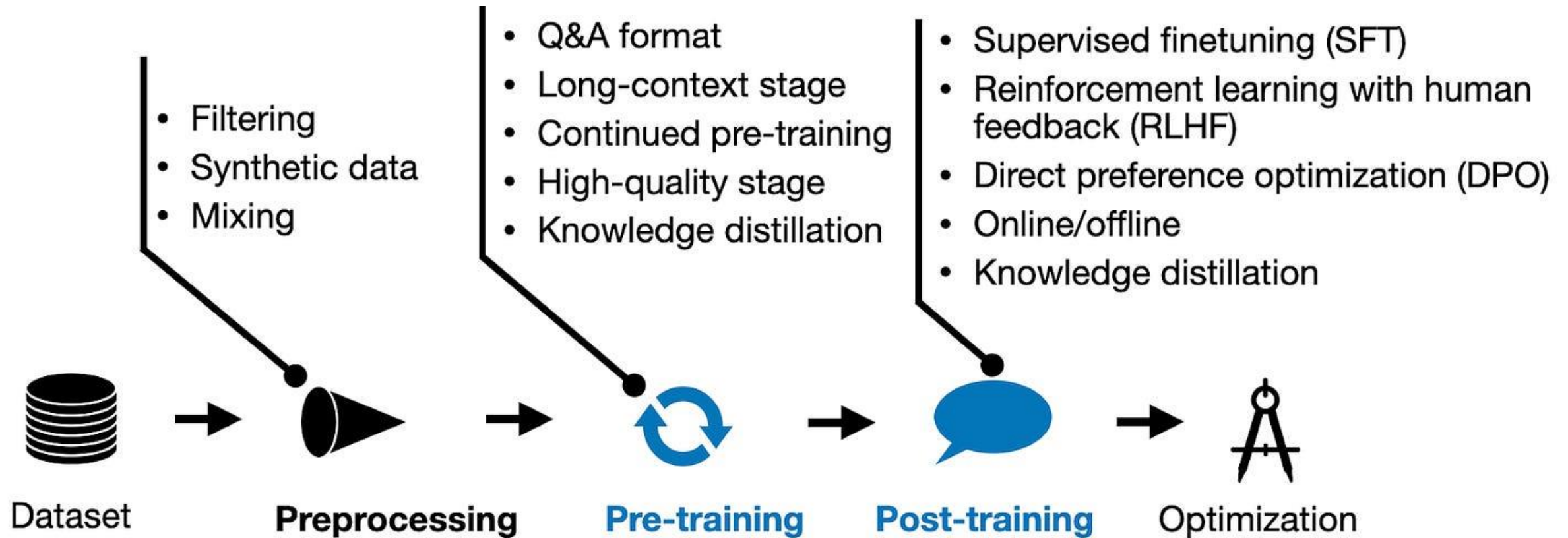


University
of Exeter

COMM117

Large Language Models and Application

Modern LLM post-training stack (2022–now)



Pretraining Objectives



BERT (Encoder-only)

Devlin et al., 2018



T5 (Encoder-decoder)

Raffel et al., 2019



Decoder-Only

The cabs ____ the same rates as those ____ by horse-drawn cabs and were ____ quite popular, ____ the Prince of Wales (the ____ King Edward VII) travelled in _____. The cabs quickly ____ known as "hummingbirds" for ____ noise made by their motors and their distinctive black and ____ livery. Passengers ____ the interior fittings were ____ when compared to ____ cabs but there ____ some complaints ____ the ____ lighting made them too ____ to those outside ____.

charged, used, initially, even, future, became, the, yellow, reported, that, luxurious, horse-drawn, were that, internal, conspicuous, cab

Original text

Thank you ~~for inviting~~ me to your party last week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>

Text: Second Law of Robotics: A robot must obey the orders given it by human beings

Generated training examples

Example #	Input (features)	Correct output (labels)
1	Second law of robotics :	a
2	Second law of robotics : a	robot
3	Second law of robotics : a robot	must
...		

Masked token prediction

Denoising span-mask prediction

Next Token Prediction

Pretraining Objectives

Problem Definition

Input: raw text $\rightarrow$ tokens $x_{1:T}$

Output: model $p_{\theta}(\cdot)$

- **Autoregressive factorization (decoder-only)**

$$p_{\theta}(x_{1:T}) = \prod_{t=1}^T p_{\theta}(x_t | x_{<t})$$

Whole-sequence probability = multiply the conditional probabilities at each step

Cross-entropy in LLM

Move from classes → next tokens

Next token prediction as classification task: Each potential word in the model's vocabulary acts as a separate class, and the model's task is to classify which word comes next given the context. → **more complex!**

cross entropy!

$$L_t = -\log p_{\theta}(y_t | x_{<t})$$

negative log-probability of the correct next token

Training minimizes average loss over all tokens in the batch:

$$L = \frac{1}{BN} \sum_{b,t} L_{b,t}$$

Pretraining Objectives- Masked Language Modelling (MLM)



Solution:

Mask subset M , predict masked tokens using bidirectional context

$$\mathcal{L}_{MLM} = - \sum_{t \in M} \log p_{\theta}(x_t \mid x_{\setminus M})$$

store gallon
↑ ↑
the man went to [MASK] to buy a [MASK] of milk

$M = 2$

BERT MLM vs Denoising (Span Corruption)

From BERT MLM to Denoising Objectives

Denoising / Span Corruption (Encoder–Decoder)

Replace **different-length spans** from the input with unique placeholders (e.g., `<extra_id_0>`); decode out the masked spans

Natural fit for **seq2seq generation**

Denoising / Span Corruption (Encoder–Decoder)

- Input: **corrupted text** (spans removed/replaced)
- Output: **generate the missing spans / original text**
- Objective:

$$\mathcal{L}_{denoise} = -\log p(x | \tilde{x})$$

Span corruption (span-level, T5-style)

The capital of France is `<extra_id_0>`.

"`<extra_id_0>` Paris"

Model family	Common pretraining objective	Self-attention mask	Cross-attention	Typical use
Encoder-only (BERT-style)	MLM (masked token prediction)	Padding mask only (bidirectional)	N/A	representation learning, classification, retrieval
Decoder-only (GPT/LLM-style)	AR (next-token prediction)	Causal mask + padding mask	N/A	text generation, chat, code
Encoder-decoder (T5/BART-style)	Denoising / span corruption (seq2seq)	Encoder: padding only ; Decoder: causal + padding	Yes: encoder outputs attended by decoder (padding mask on encoder)	translation, summarization, conditional generation

Introduction Today

1. Scaling law: How do we spend compute optimally?
2. Optimisation
3. Parallelism: How do we fit/train huge models efficiently?

Scaling Laws

- LLM ideas are not new
- The breakthrough is **scale**
- Three things **scaled together**:
 - Data
 - Compute
 - Architecture

Nothing here is magic — it's **scale** + the right **inductive bias**.

Scaling Laws

Scaling laws describe how the **performance** of large language models (LLMs) improves predictably as you **increase certain factors**.

- **parameters (N)**: the size of the model (measured in parameters),
- **data/tokens (D)**: the size of the training dataset,
- **compute (C)**: the amount of compute used for training.

change N, D, or C by a factor → loss/performance movement

Scaling Laws

[Training Compute-Optimal Large Language Models](#) (DeepMind 2022)

We investigate the **optimal model size and number of tokens** for training a transformer language model **under a given compute budget**. We find that current large language models are significantly under-trained, a consequence of the recent focus on scaling language models whilst keeping the amount of training data constant. By training over 400 language models ranging from 70 million to over 16 billion parameters on 5 to 500 billion tokens, **we find that for compute-optimal training, the model size and the number of training tokens should be scaled equally: for every doubling of model size the number of training tokens should also be doubled**. We test this hypothesis by training a predicted compute-optimal model, *Chinchilla*, that uses the same compute budget as *Gopher* but with **70B parameters and 4× more more data**. *Chinchilla* uniformly and significantly outperforms *Gopher* (280B), GPT-3 (175B), Jurassic-1 (178B), and Megatron-Turing NLG (530B) on a large range of downstream evaluation tasks. This also means that *Chinchilla* **uses substantially less compute for fine-tuning and inference**, greatly facilitating downstream usage. As a highlight, *Chinchilla* reaches a state-of-the-art average accuracy of 67.5% on the MMLU benchmark, greater than a 7% improvement over *Gopher*.

FLOPs

- A **floating-point operation** is a basic arithmetic step on real-valued numbers, like **addition**, **multiplication**, **fused multiply-add**, etc., done in floating-point formats (fp32, fp16, bf16).

FLOPs  a count of such operations (e.g., “this training run took 3×10^{20} FLOPs”).

Why it matters in LLM training

Training cost is dominated by **matrix multiplications in transformers**, so we often **measure the compute budget in FLOPs**.

- **More parameters (N)** → more FLOPs per token
- **More tokens (D)** → more total FLOPs



If your model needs **$\sim k$ FLOPs** to process **one token**, then:

- Training on D tokens costs about $k \times$

D FLOPs

Undertrained vs Overtrained – Optimal Training

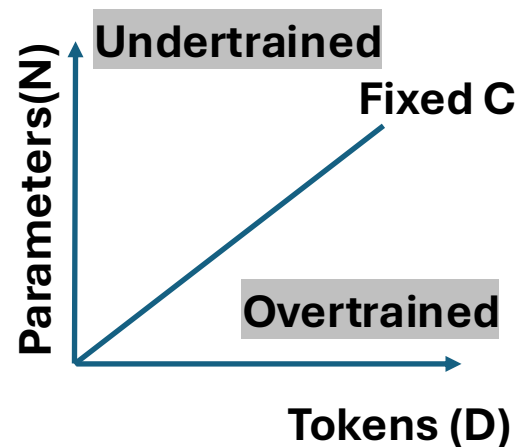
Fixed compute budget: Training FLOPs $C \propto N \times D$

Undertrained

Too **few** tokens for model size

- Training loss is higher than it could be for this compute
- Model capacity not fully used (“data-starved”)
- Scaling parameters $\uparrow$ without scaling tokens $\uparrow$

Fix: train on **more tokens** or use a **smaller model**



Overtrained

Too **many** tokens for model size

- Diminishing returns: extra tokens add little improvement
- Model becomes “capacity-limited”
- Huge token budget on a small model

Fix: increase **model size** (or model quality/architecture)

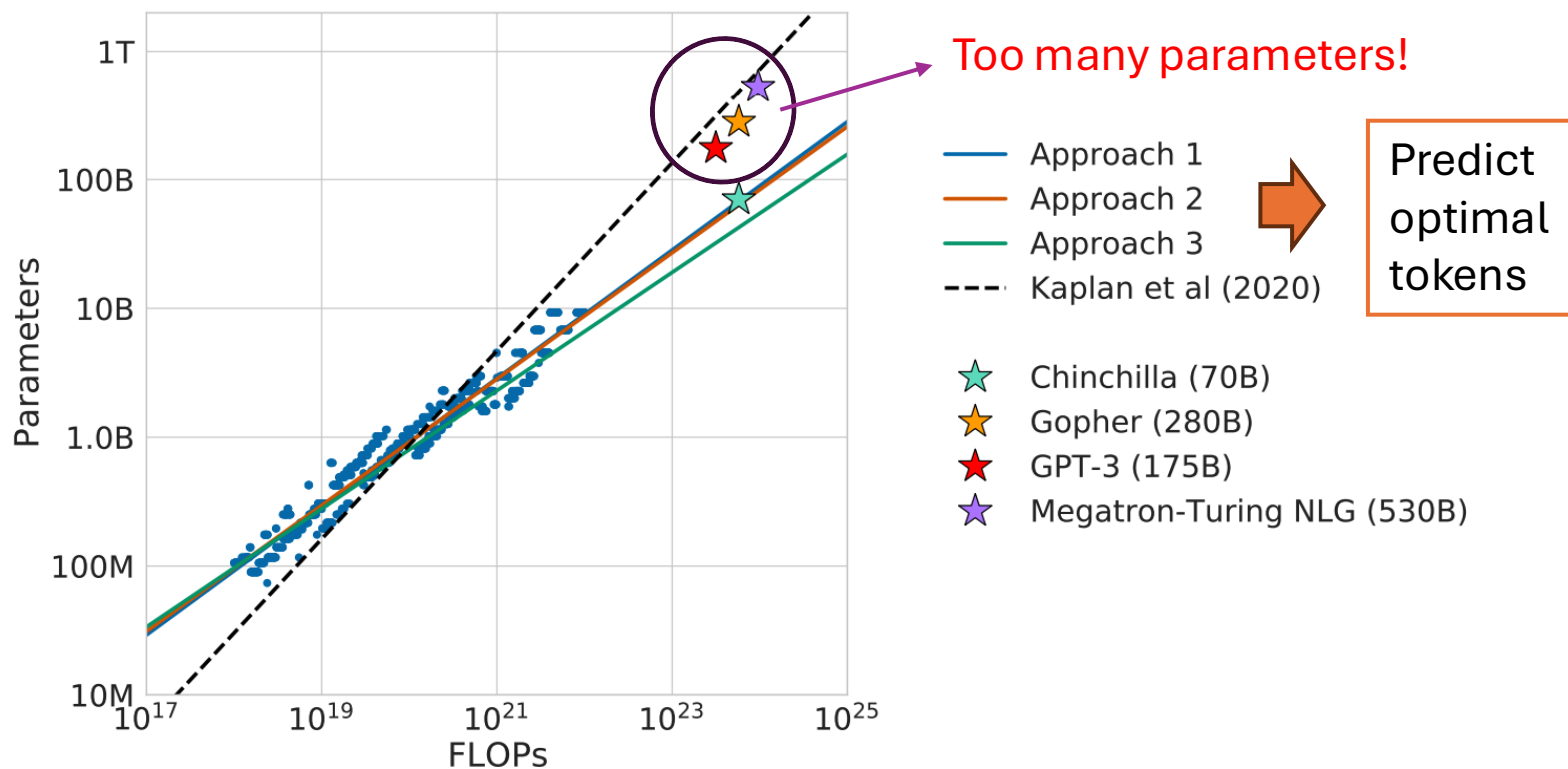
Optimal Training

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
Chinchilla	70 Billion	1.4 Trillion

Fixed compute budget: Training FLOPs $\mathcal{C} \propto N \times D$

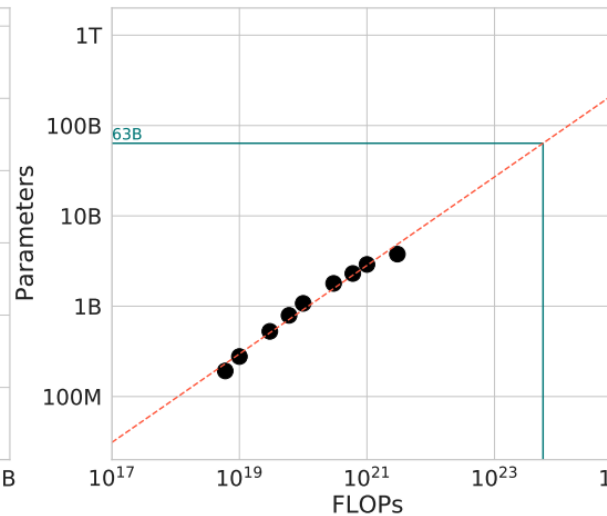
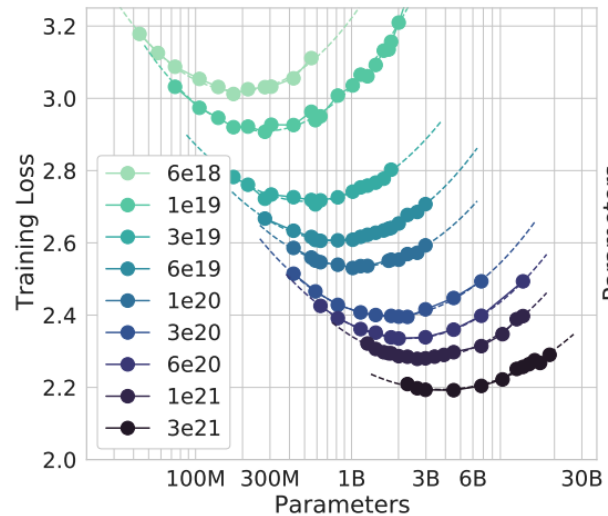


compute-optimal trade-off between N and D

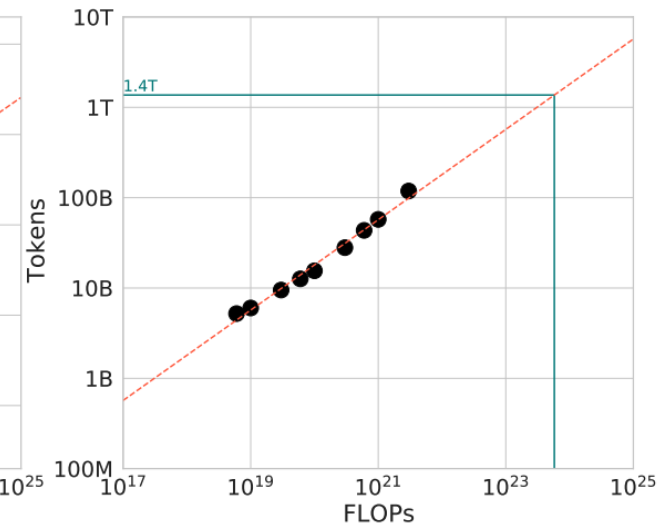


Paper Adds

- Many earlier large models were **undertrained** (too few tokens for their size)
- Compute-optimal training typically uses **more tokens than previously assumed**
- Practical takeaway: for a fixed compute budget, **it's often better to train a smaller model longer than a larger model briefly**



compute-optimal
parameters scale with
compute



compute-optimal
tokens scale with
compute

Limits & how to use scaling laws responsibly

- Scaling laws **depends on** model design, data distribution, objective
- **Data quality** and contamination can break the trend
- Post-training changes behavior; scaling laws mainly describe **pretraining loss**

Scaling laws are not universal laws of nature
they're empirical patterns that hold *for a particular model family, dataset mix, and training objective*—change those, and the curve moves.

Optimisation

Why optimization is harder at scale:

Not bad loss → Weird loss

- **Loss spikes** (sudden jump, then maybe recover)
- **Divergence / NaNs** (loss becomes NaN/Inf)
- **Training stalls** (loss plateaus unexpectedly)
- **Silent regression** (loss looks fine, downstream behavior worsens)
- **Instability after scaling** (same code, more GPUs → worse training)

Constraints: **memory, throughput, numeric, communication**

Optimisation

Why optimization is harder at scale:

Not bad loss → Weird loss

- **Loss spikes** (sudden jump, then maybe recover)
- **Divergence / NaNs** (loss becomes NaN/Inf)
- **Training stalls** (loss plateaus unexpectedly)
- **Silent regression** (loss looks fine, downstream behavior worsens)
- **Instability after scaling** (same code, more GPUs → worse training)

Constraints: **memory, throughput, numeric, communication**

Adam vs AdamW

Adam adapts learning rates per-parameter using moments of gradients

- Use gradient: $g_t \leftarrow g_t + \lambda \theta_t$
- Then Adam update uses the modified gradient
- Result: the “decay” is **scaled by adaptive moments**, not uniform

AdamW (decoupled weight decay)

- Adam step uses raw gradient g_t
- **Separate weight decay step:** $\theta_{t+1} \leftarrow (1 - \eta\lambda)\theta_t - \text{AdamStep}(g_t)$
- Result: decay is **consistent** across parameters

LLMs: AdamW + warmup + decay + grad clipping

Effective batch size

Effective batch size is what the optimizer “sees”

- $B_{eff} = B_{micro} \times grad_accum \times \#GPUs$
- Same code, more GPUs $\Rightarrow$ larger B_{eff}
- Bigger B_{eff} $\neq$ free speed; it changes training dynamics

Big batch reduces noise — not always good

- Small batch: noisy gradients $\rightarrow$ implicit regularization
- Big batch: stable updates but can worsen generalization

 Larger B_{eff} often needs **Learning Rate schedule retuning**

LR schedule is the first stability lever

- **Peak learning rate** (max LR)
- **Warmup length** (number of steps/tokens to ramp up LR)
- **Decay shape** (cosine / linear / exponential)
- **Total training steps** (schedule length)

If you increase GPUs, B_{eff} increases, and the “same” LR can become **too aggressive** (or sometimes too conservative), causing:

- loss spikes / divergence, or
- slower convergence

We scaled from 8 GPUs to 64 GPUs → effective batch increased 8× → we **retuned peak LR and warmup**.

Gradient clipping: the seatbelt

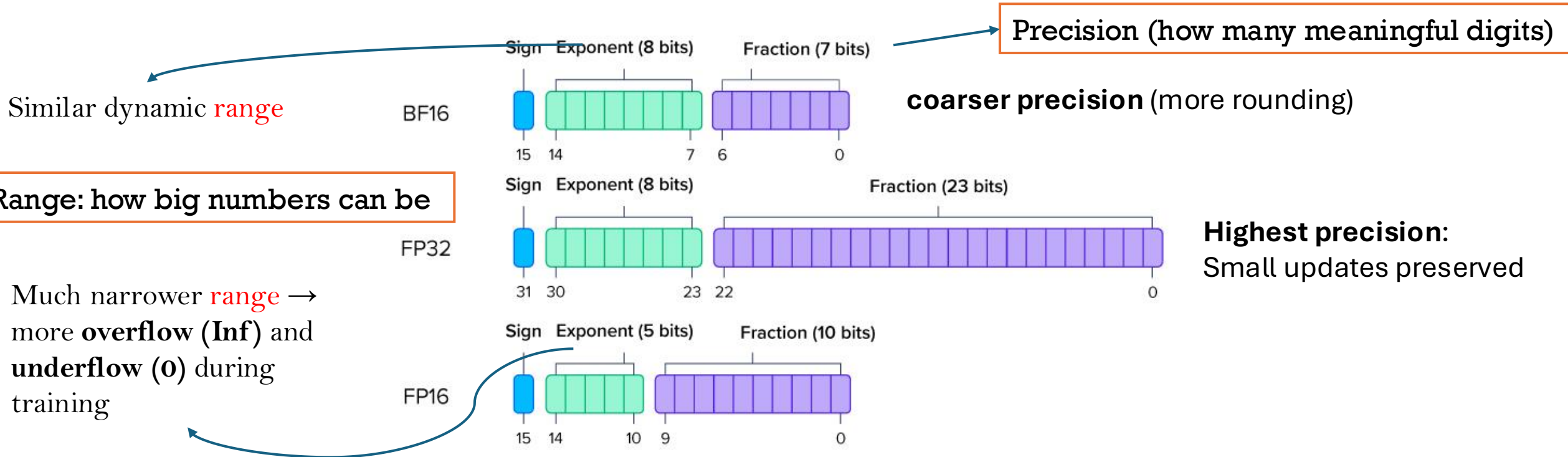
- **Clip global grad norm** (common in LLM training)
- Useful against outlier batches and long-context instability
- If always clipping → LR too high / batch too large / data noisy

When B_{eff} changes, retune:

- **peak LR**, then
- **warmup steps**, then
- **grad clipping** (as a stabilizer)

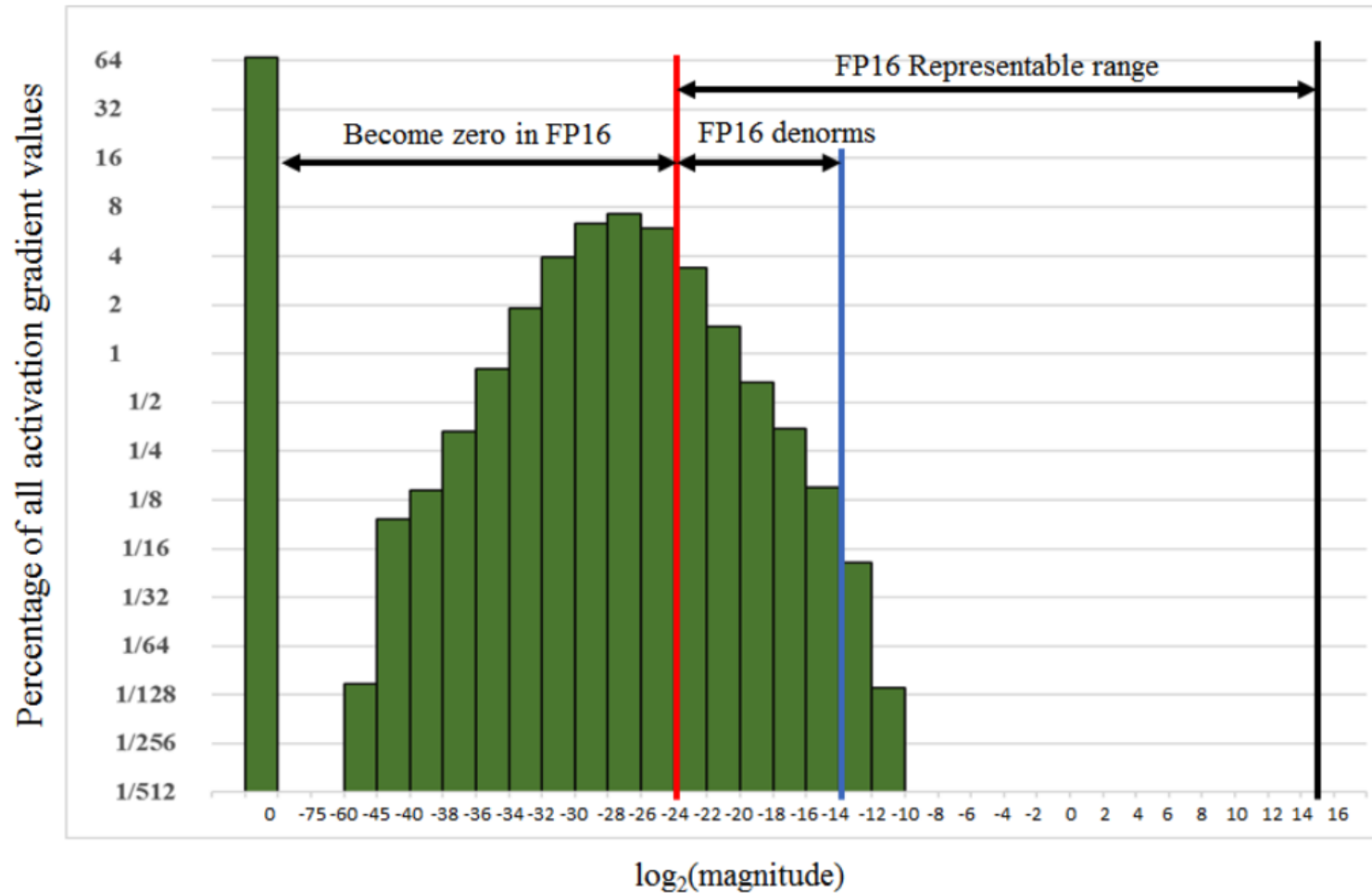
Scaling Challenge: Numerical Types

Floating point formats supported by acceleration hardware



- **BF16** originated in Google's ecosystem and was used on **TPU v2 (2017)** for training.
- **NVIDIA GPUs pre-Ampere** were mainly **FP32 + FP16 (Tensor Cores)** for deep learning training.
- **NVIDIA A100 (Ampere, 2020)** brought **BF16 Tensor Core** support to GPUs.

Numerical Types



Neural network prefer **bigger range** than better precision

MIXED PRECISION TRAINING

Using **different numerical types** at different part of the training process

- Parameters, activations, and gradients often use **FP16**
- Optimizer states often needs **FP32**

Maintaining main copies of FP32 for calculations

Dynamically scaling up loss to fit gradients etc. in FP16 range

FP16 has much lower precision
→ small updates can get **rounded to zero**

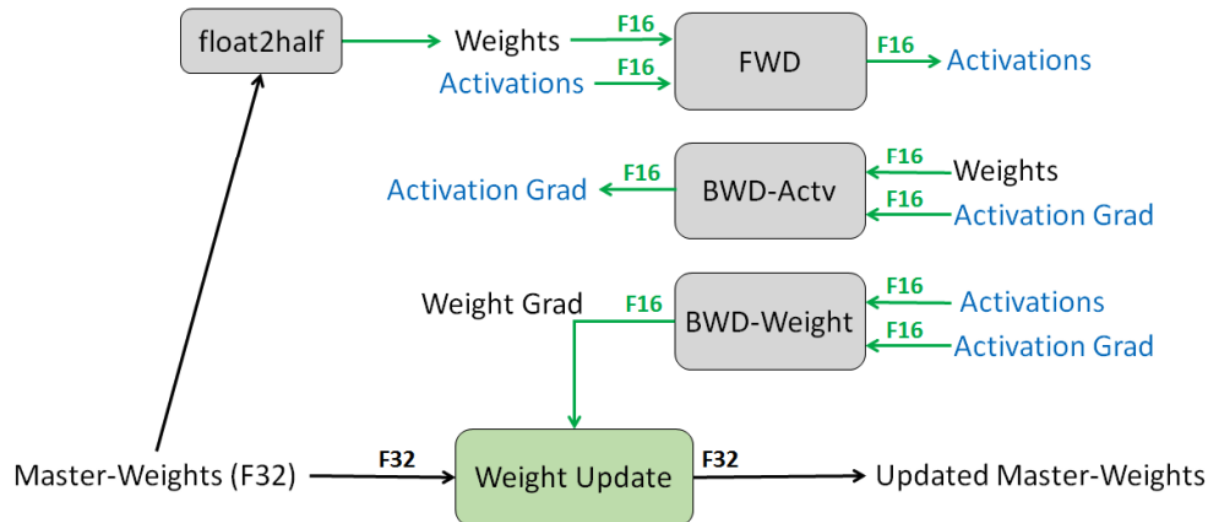


Figure 1: Mixed precision training iteration for a layer.

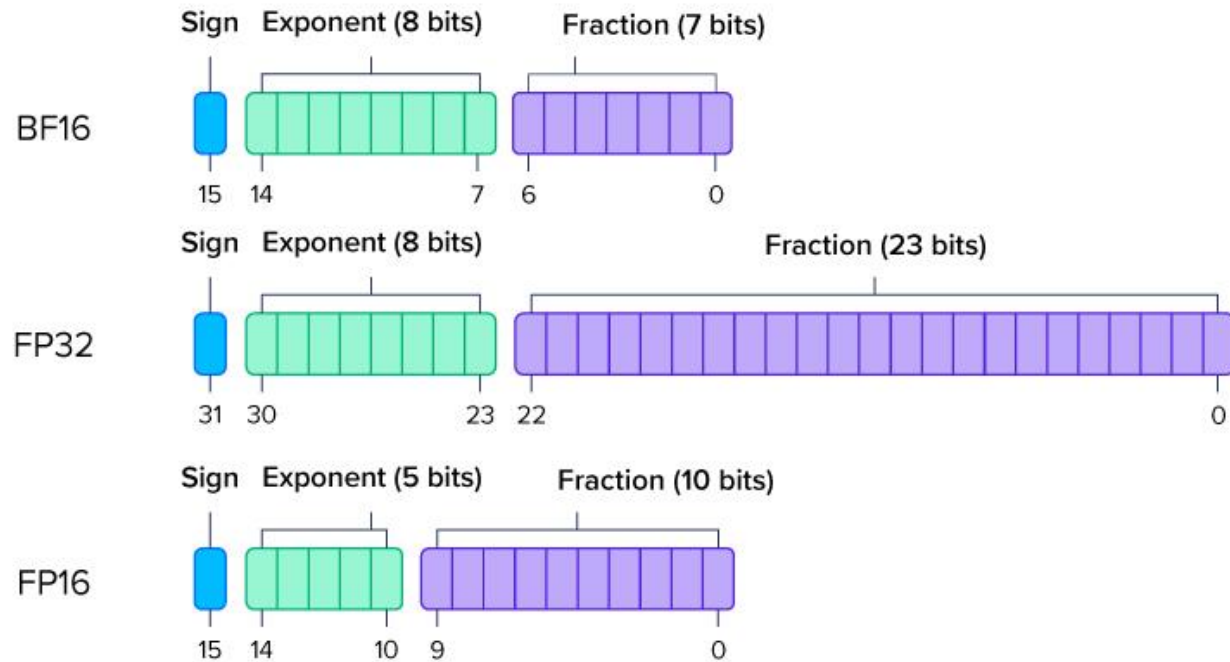
multiply the loss by a scale factor S before backprop:

- Compute: $L' = S \cdot L$
- Backprop gives gradients multiplied by S : $g' = S \cdot g$
- Then before the optimizer step, divide gradients back:

$$g = g' / S$$

BF16

BF16 is the preferred numerical type on A100 and H100



- Same range as FP32: reduced the needs for mixed precision training while being way more stable
- Coarse precision (shorter fraction): mostly fine, only a few places in neural network need more fine-grained precision

But: (small weight updates can disappear if too low precision) → keep FP32 for master weights

Parallel Training

Parallelism is an optimization change

As scale grows, training with one GPU is not enough. There are many ways to improve efficiency on single-GPU training

- Checkpointing: moving part of the operations to CPU memory
- Quantizing different part of the optimization to reduce GPU memory cost

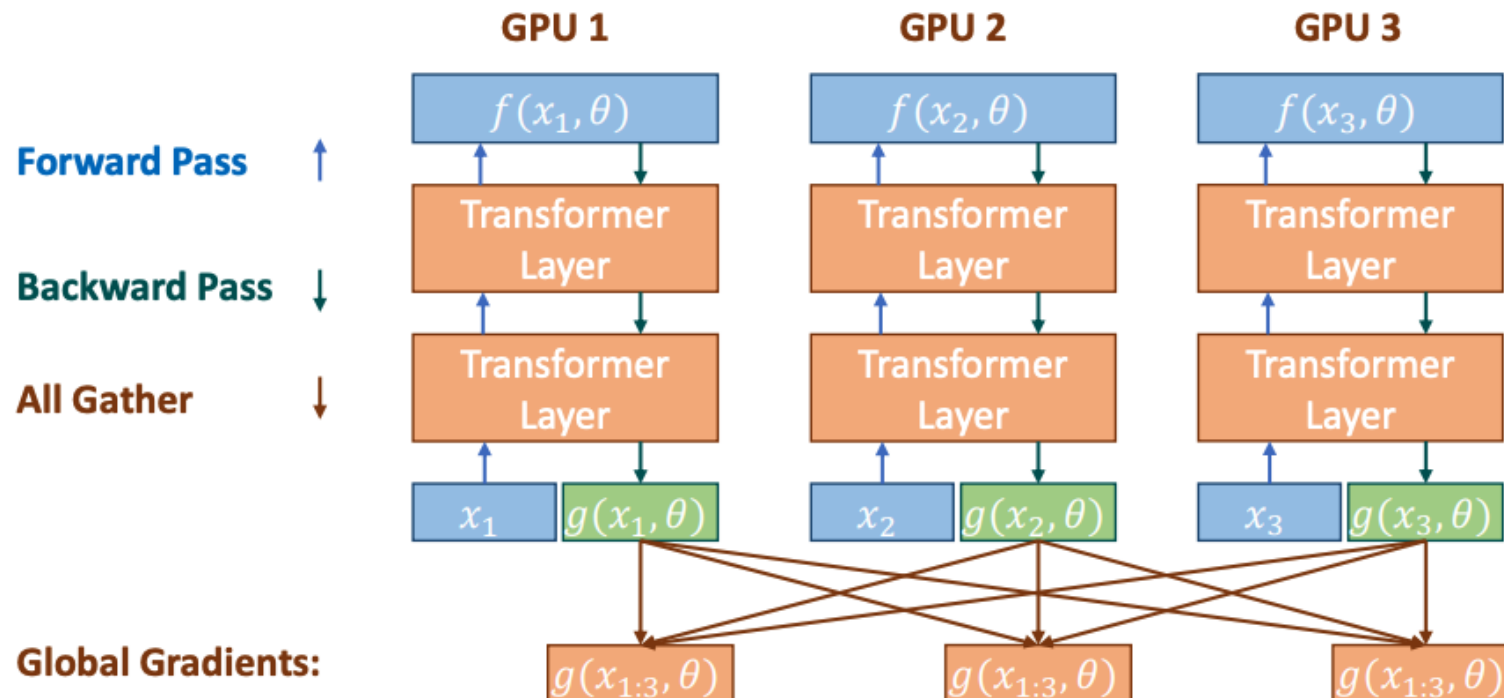
Eventually **more FLOPs are needed!**

→ Parallel Training

- When model training can fit into **single-GPU** → **Data parallelism**
- When model training **cannot fit into single-GPU** → **Model parallelism**: pipeline or tensor

Parallel Training: Data Parallelism

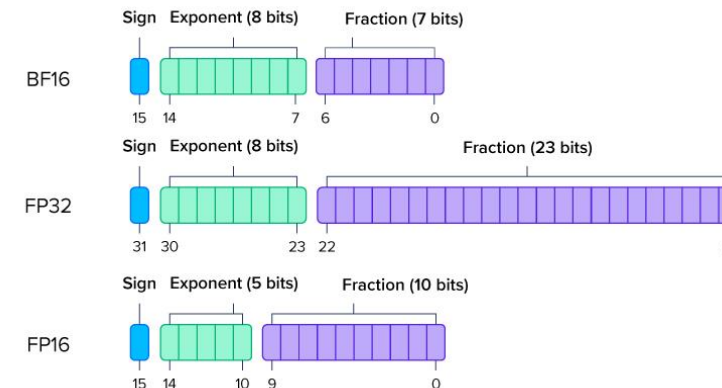
- Split training **data batch** into different GPUs
- Each GPU maintains **its own copy** of model and optimizer
- Each GPU gets a **different local data batch**, calculates its gradients
- Gather local gradients together to each GPU for **global updates**



Memory breakdown

Memory & throughput (speed) — total bits

- **BF16 and FP16 are 16-bit**
about **half the memory** of FP32
higher throughput on modern accelerators (Tensor Cores/TPUs)
- **FP32 is 32-bit**
slower / more memory bandwidth



Component (per parameter)	Typical dtype	Bytes
Weights (model)	BF16/FP16	2
Gradients	BF16/FP16	2
Adam m (1st moment)	FP32	4
Adam v (2nd moment)	FP32	4
FP32 master weights (<i>often</i>)	FP32	4
Total		~16 bytes / param

Adam needs two extra FP32 buffers (m and v), which is the ‘tax’.”

Memory breakdown

LLM size grew quickly and passed the limit of single GPU memory

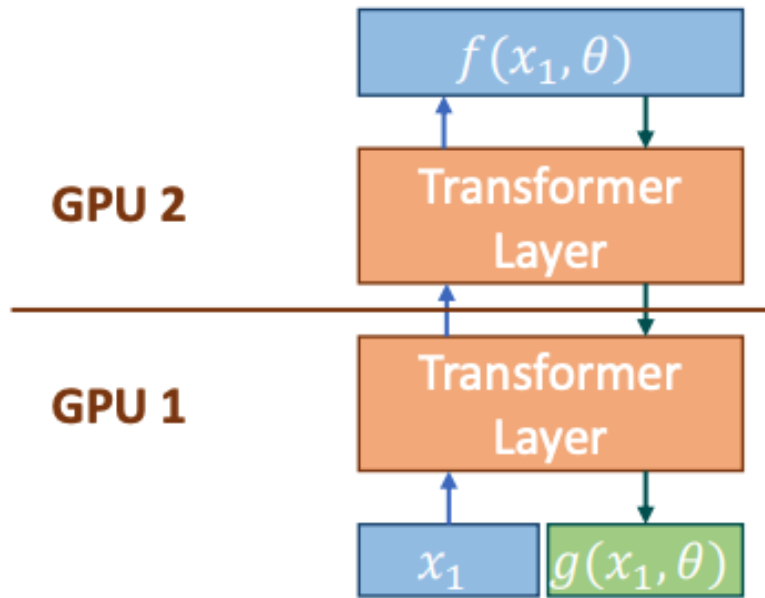
	Cost of 10B Model	Function to parameter count (Ψ)
Parameter Bytes	20GB	2Ψ
Gradient Bytes	20GB	2Ψ
Optimizer State: 1st Order Momentum	20GB	2Ψ
Optimizer State: 2nd Order Momentum	20GB	2Ψ
Total Per Model Instance	80GB	8Ψ

Table 1: Memory Consumption of Training Solely with BF16 (Ideal case) of a model sized Ψ

Solution: Split network parameters (thus their gradients and corresponding optimizer states) to different GPUs

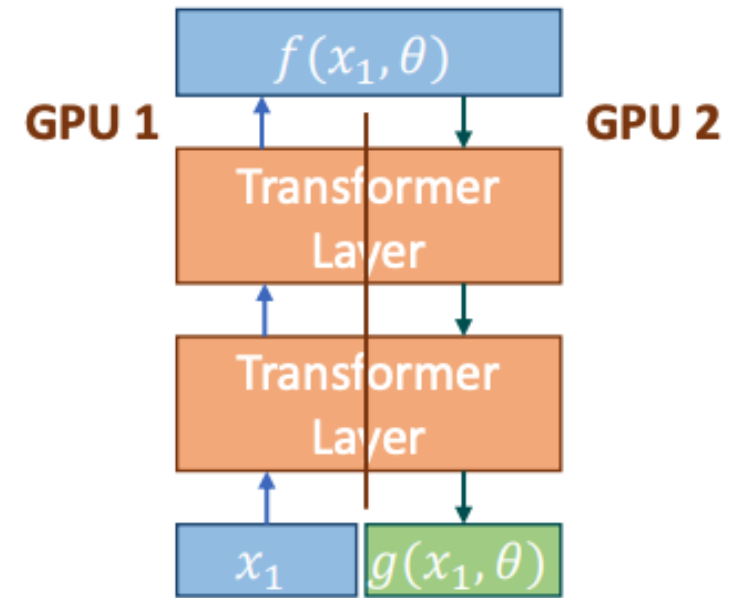
Parallel Training: Model Parallelism

Pipeline Parallelism



Split by Layers

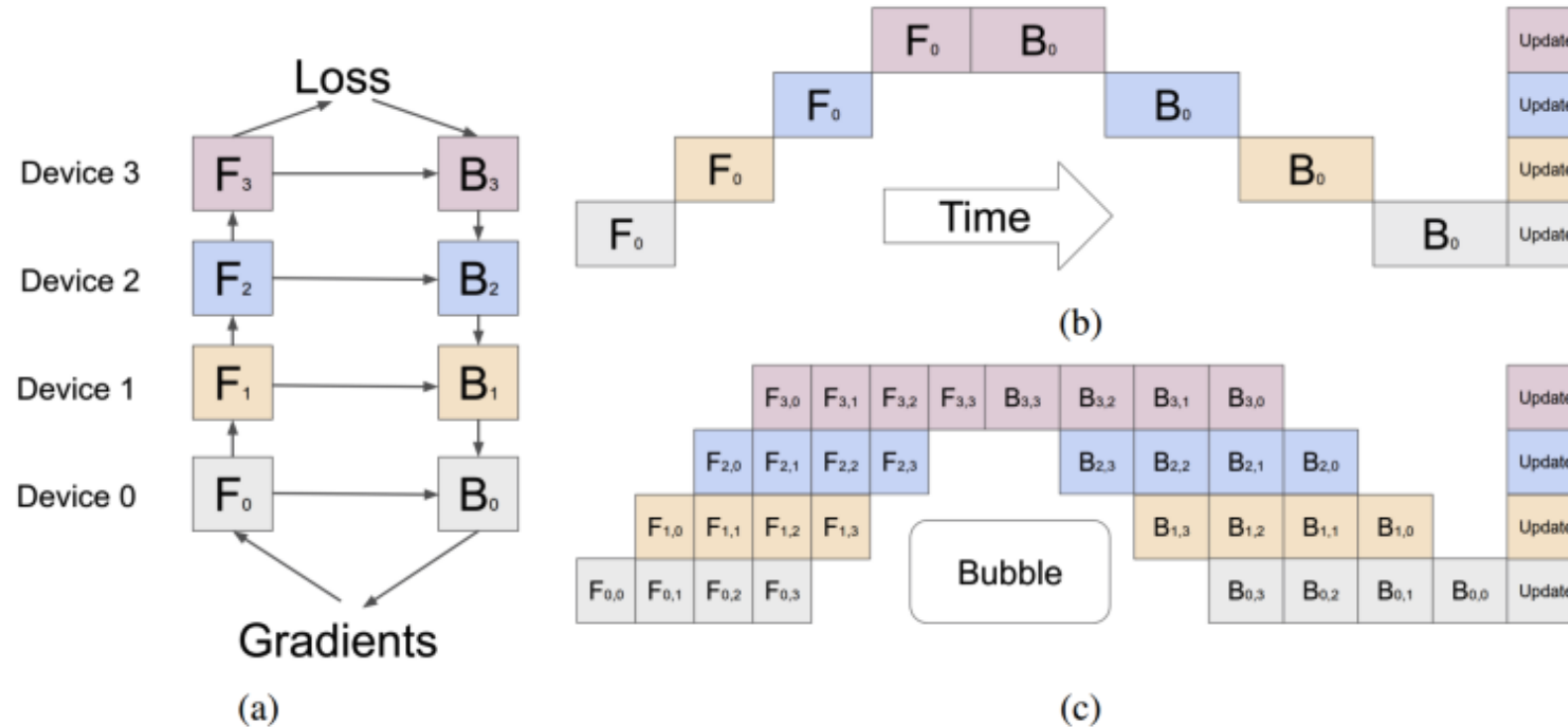
Tensor Parallelism



Split Tensors

Model Parallelism- Pipeline Parallelism

Split network by layers, aligning devices by layer order to a pipeline, and pass data through devices



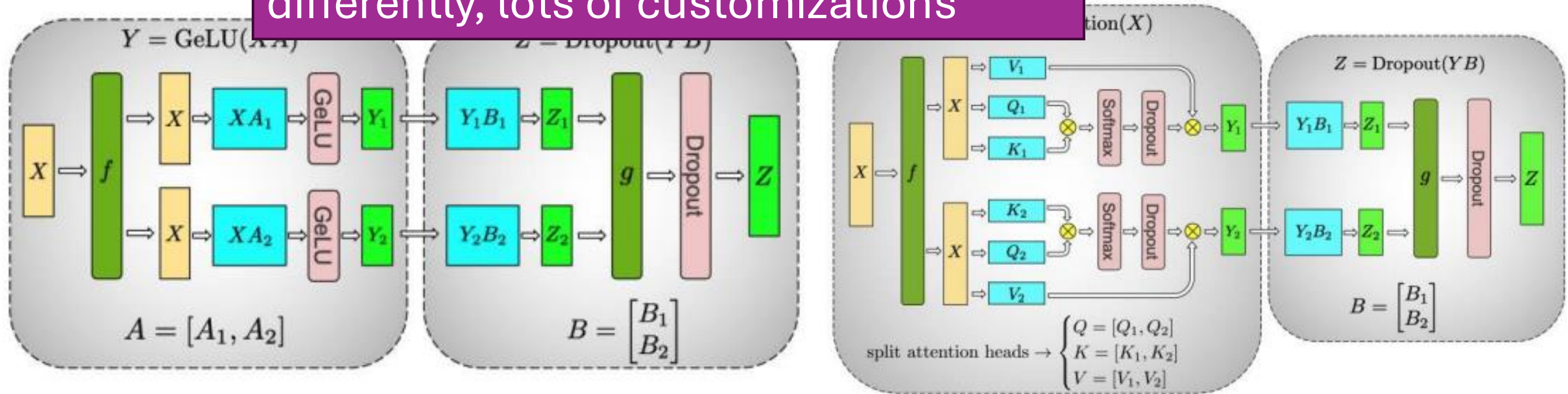
- **Activations** between nearby devices in forward pass
- **Partial gradients** between nearby devices in backward
- **Full gradients** from Device 0 to All others

Model Parallelism- Tensor Parallelism

Pros: No bubble

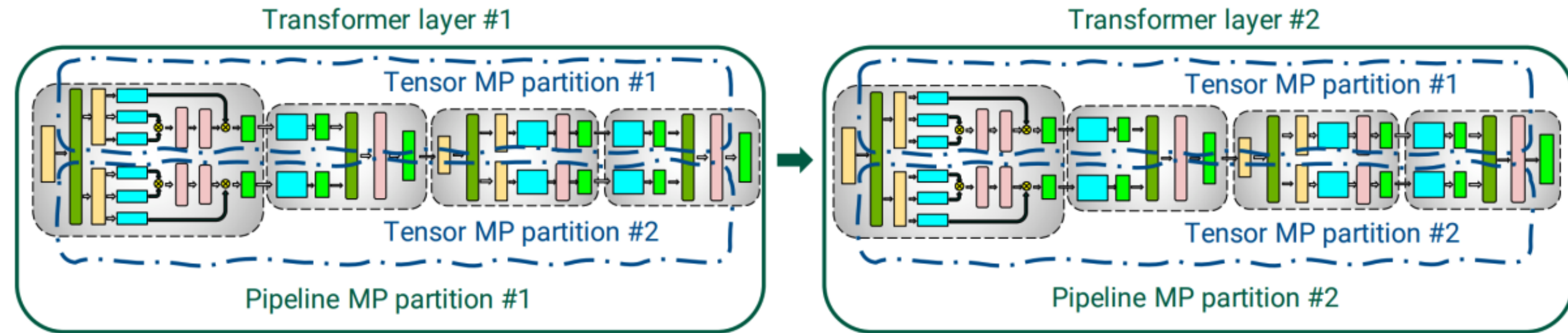
Split the p

Cons: Different blocks are better split differently, lots of customizations

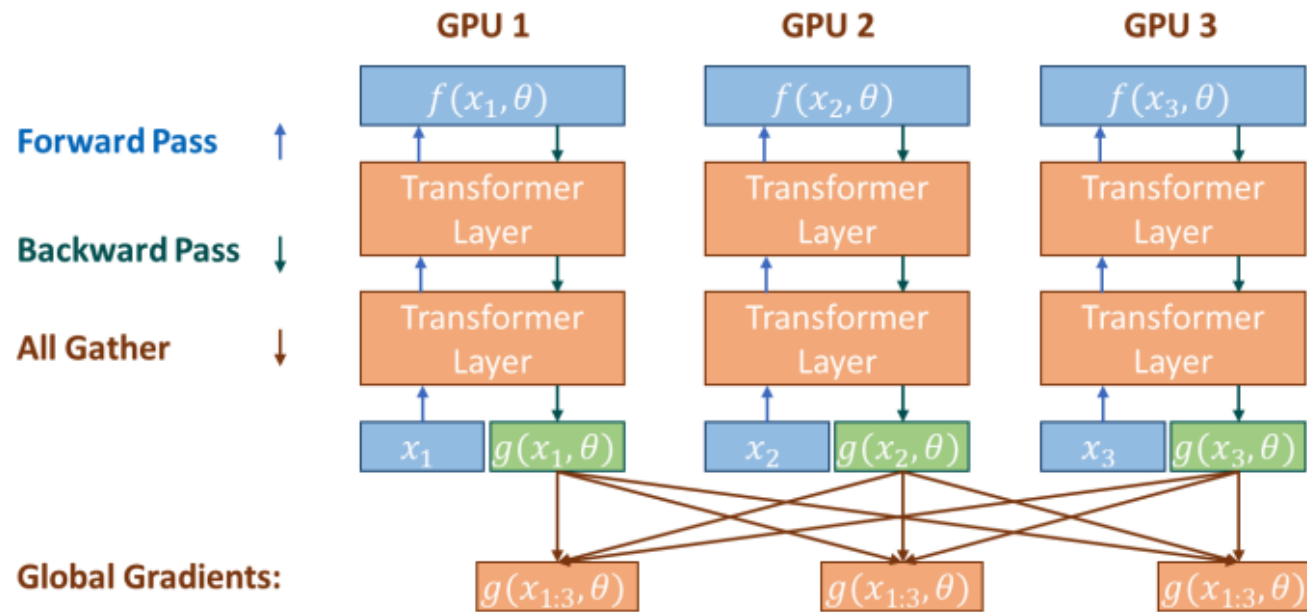


Training Parallelism- All Together

Often data parallelism and model parallelism are used together
Pipeline Parallelism and Tensor Parallelism can also be used together.



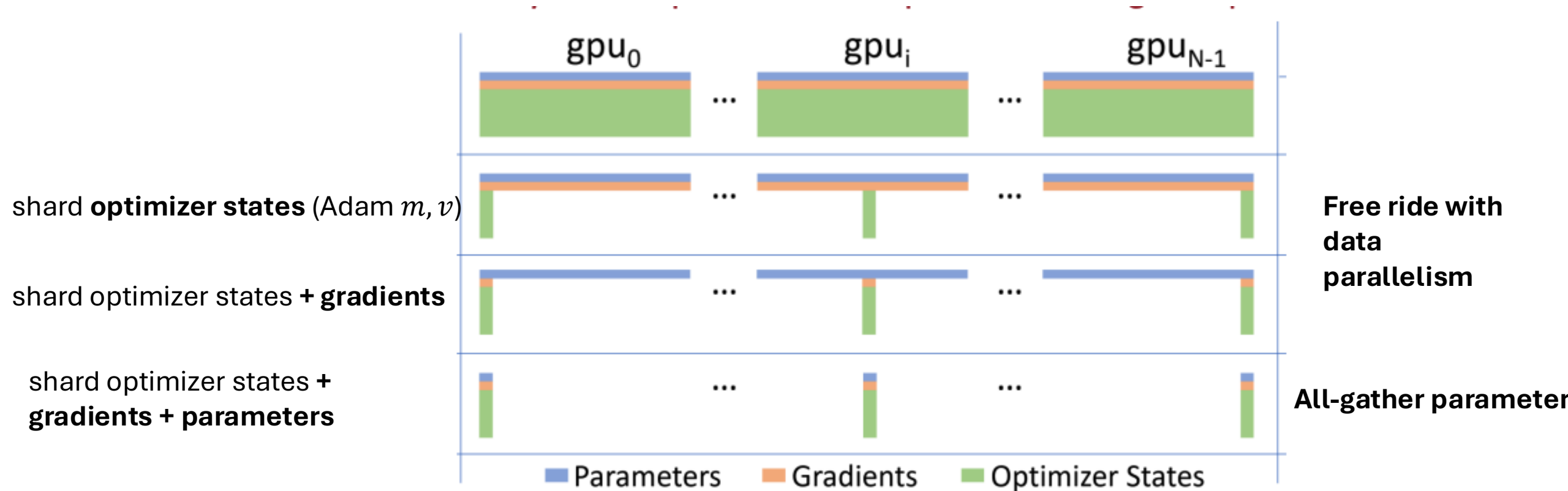
ZeRO (DeepSpeed): family of sharding techniques



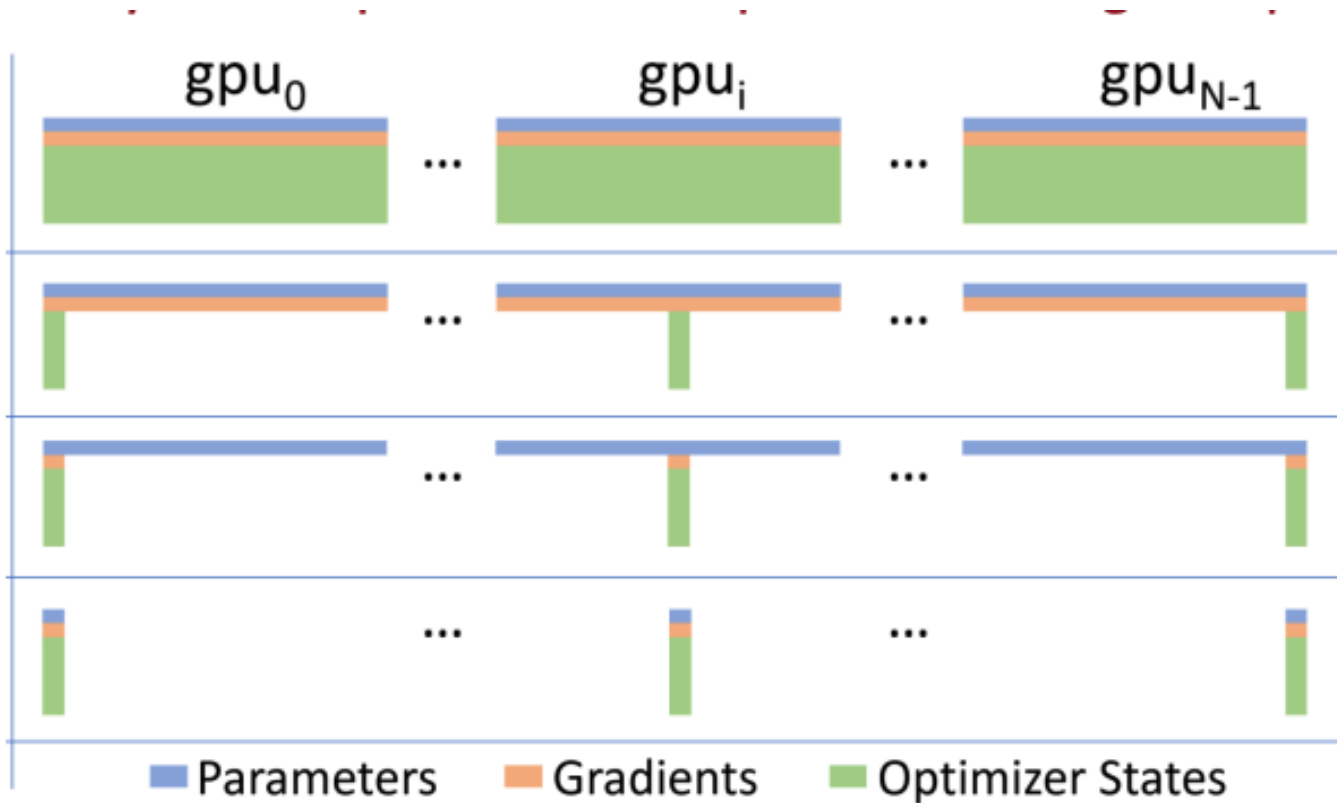
- In data parallelism, each device **only has access to local gradient**
- All gather operation required on all gradients anyway

Remove redundancy in Data Parallelism by **sharding training states** across GPUs.

ZeRO (DeepSpeed): family of sharding techniques



ZeRO (DeepSpeed): family of sharding techniques



What you gain

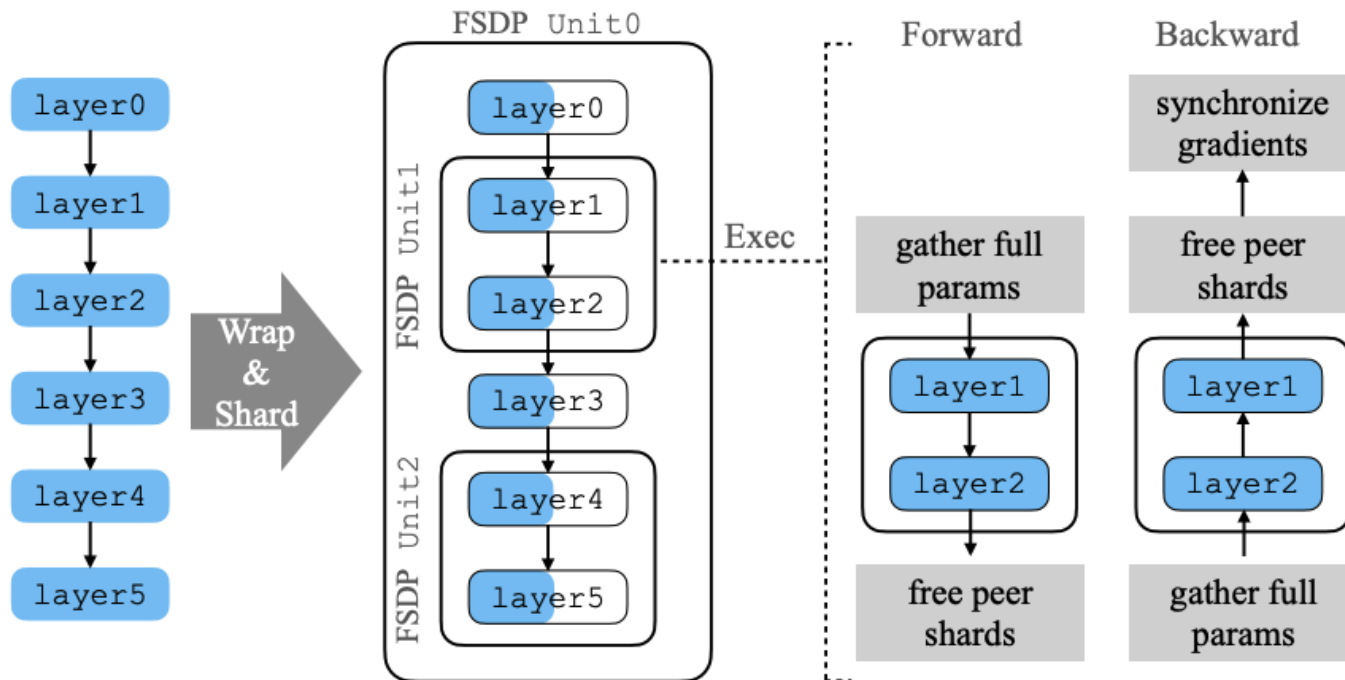
- Much lower **per-GPU memory** → train bigger models / larger batch

What you pay

- More **communication** (all-gather params, reduce-scatter grads)
- More runtime complexity (partitioning, scheduling)

FSDP (PyTorch): Fully Sharded Data Parallel

A PyTorch-native implementation of **parameter + grad + optimizer sharding** (i.e., *ZeRO-3-style*).



- **Params are sharded** across GPUs at rest
- During compute:
- **All-gather** full params for a module just-in-time
 - Run **forward/backward**
 - **Reduce-scatter** grads
 - Re-shard / free full params

Figure 1: FSDP Algorithm Overview

Other literatures about Scaling Up

- **Different configurations of layer normalization: pre layernorm, post layernorm and their combination**
 - Xiong et al. “On Layer Normalization in the Transformer Architecture”. ICML 2020
 - Zhang and Sennrich. “Root Mean Square Layer Normalization”. NeurIPS 2019
- **Position embeddings for longer contexts and expressiveness**
 - Su et al. “Roformer: Enhanced transformer with rotary position embedding.” arXiv 2021
- **Stability improvement from adaptive initialization**
 - Liu et al. “Understanding the Difficulty of Training Transformers”. EMNLP 2020

Next steps

- Post training LLM I:
SFT + RLHF+ eval + finetuning pipeline

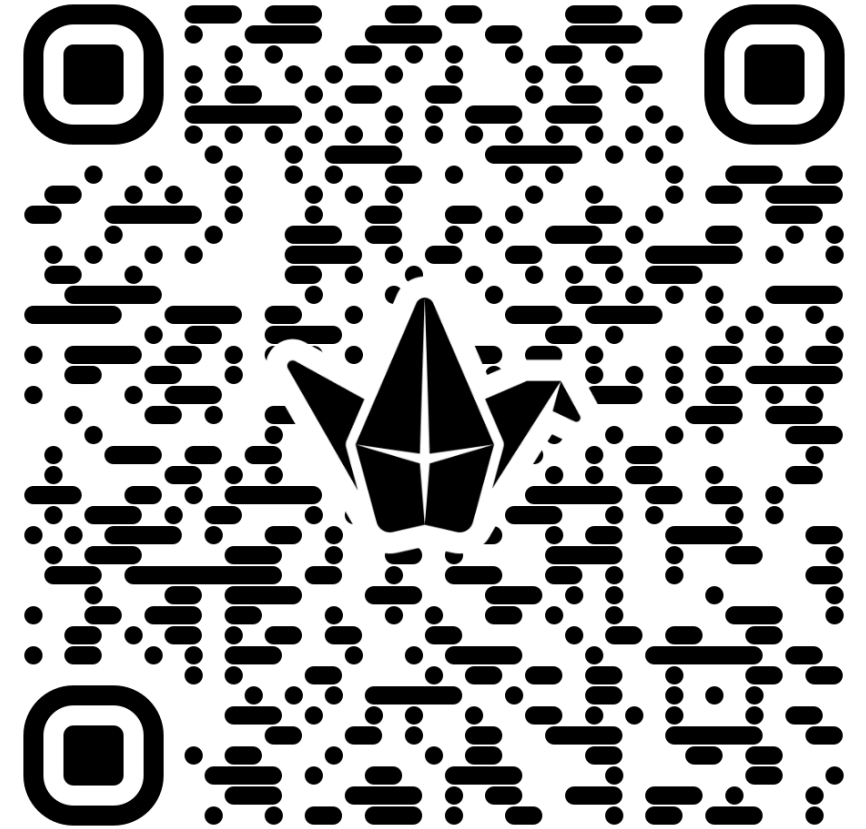
Task:

Read This Paper

Will explain it Next Week!

Input your thoughts/questions in Padlet

[Understanding the Difficulty of Training Transformers](#)



Thank you!

If you want to ask us any questions about the module, the university, or anything else, drop us an email.

Pro tip: write **COMM117** on the email subject line, to make sure we give it priority.

 r.mu2@exeter.ac.uk

Questions?